

LLM Basics for Healthcare Professionals

A simple, hands-on workshop with fictional healthcare examples

This notebook explains how large language models (LLMs) work using plain language, simple diagrams, and small Python examples.

Learning goals

1. See how text becomes tokens and numbers.
2. Understand parameters, embeddings, attention, and next-token prediction.
3. See how training reduces errors.
4. Explore retrieval-augmented generation (RAG), agents, MCP, and A2A.
5. Practice safe, responsible use in healthcare.

Safety check: Everything here is educational. All people, notes, policies, and messages are fictional. Never use these examples for diagnosis, treatment, medication dosing, or real patient care. Never enter protected health information (PHI) into an unapproved tool.

The GenAI Landscape — Model Creators and Model Users

The LLM landscape has two broad roles: **organizations that create and train models**, such as OpenAI, Anthropic, Meta, Google, and xAI; and **people and organizations that use those models** for everyday tasks such as programming, writing, and summarizing. The same organization can play both roles.

This workshop covers both: understanding how LLMs work and practicing how to use them.

THE GenAI LANDSCAPE

Two broad roles in the world of large language models (LLMs)

01 MODEL CREATORS

Build and train the models

Organizations that develop the underlying LLMs.

OpenAI • Anthropic • Meta
Google • xAI • and others

Data → Training → Evaluation → Model

In this workshop

Tokens, embeddings, attention, and training



02 MODEL USERS

Apply models to everyday work

People and organizations using existing LLMs.

Developers • Writers • Educators
Healthcare teams • Businesses • You

Programming, writing, summarizing, and more

In this workshop

Prompting, RAG, agents, and responsible use

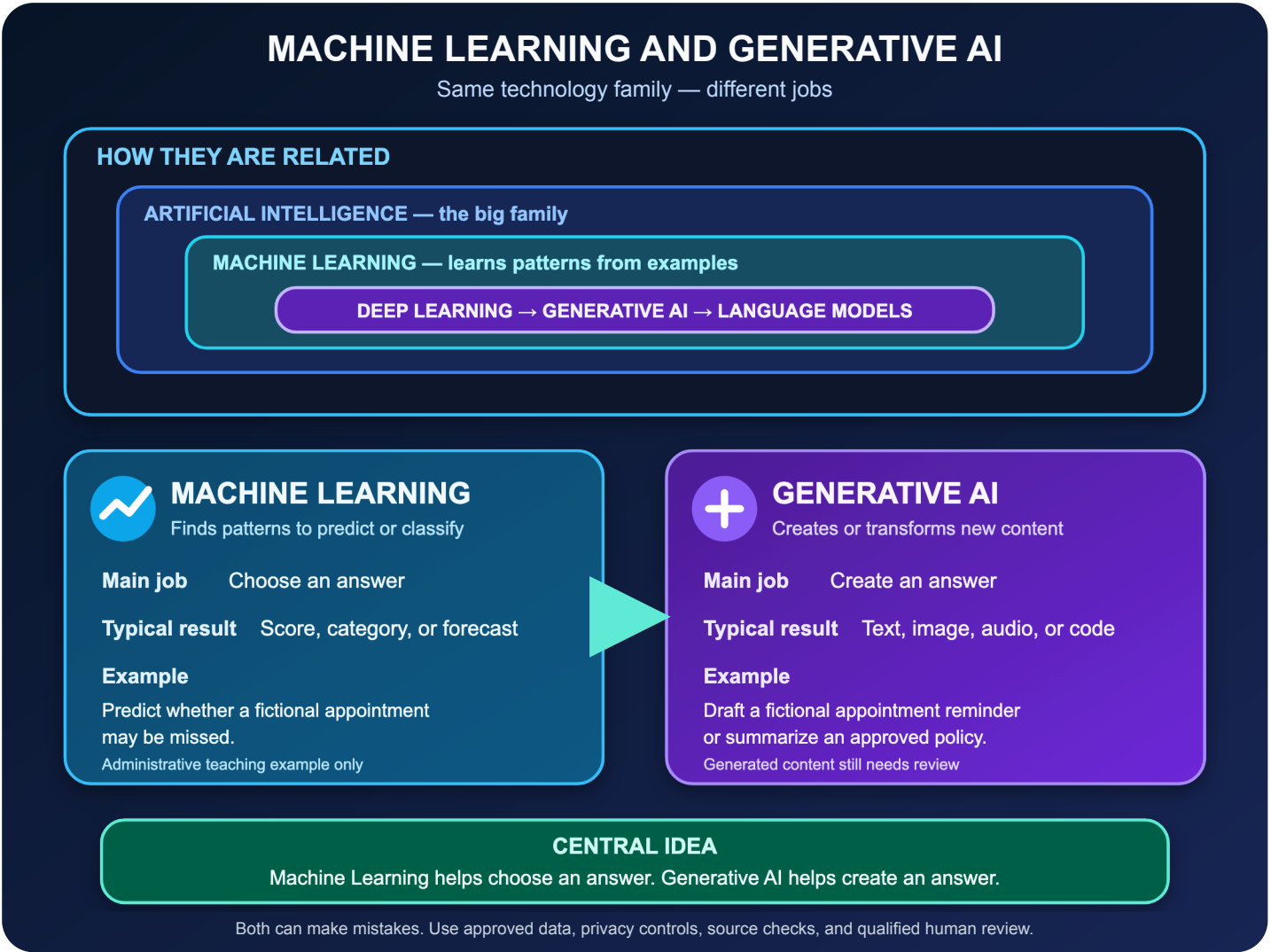
These roles can overlap: the same organization may create and use models.



THIS WORKSHOP COVERS BOTH
Understand how LLMs work + Practice how to use them

A simplified, LLM-focused view of the broader generative AI landscape

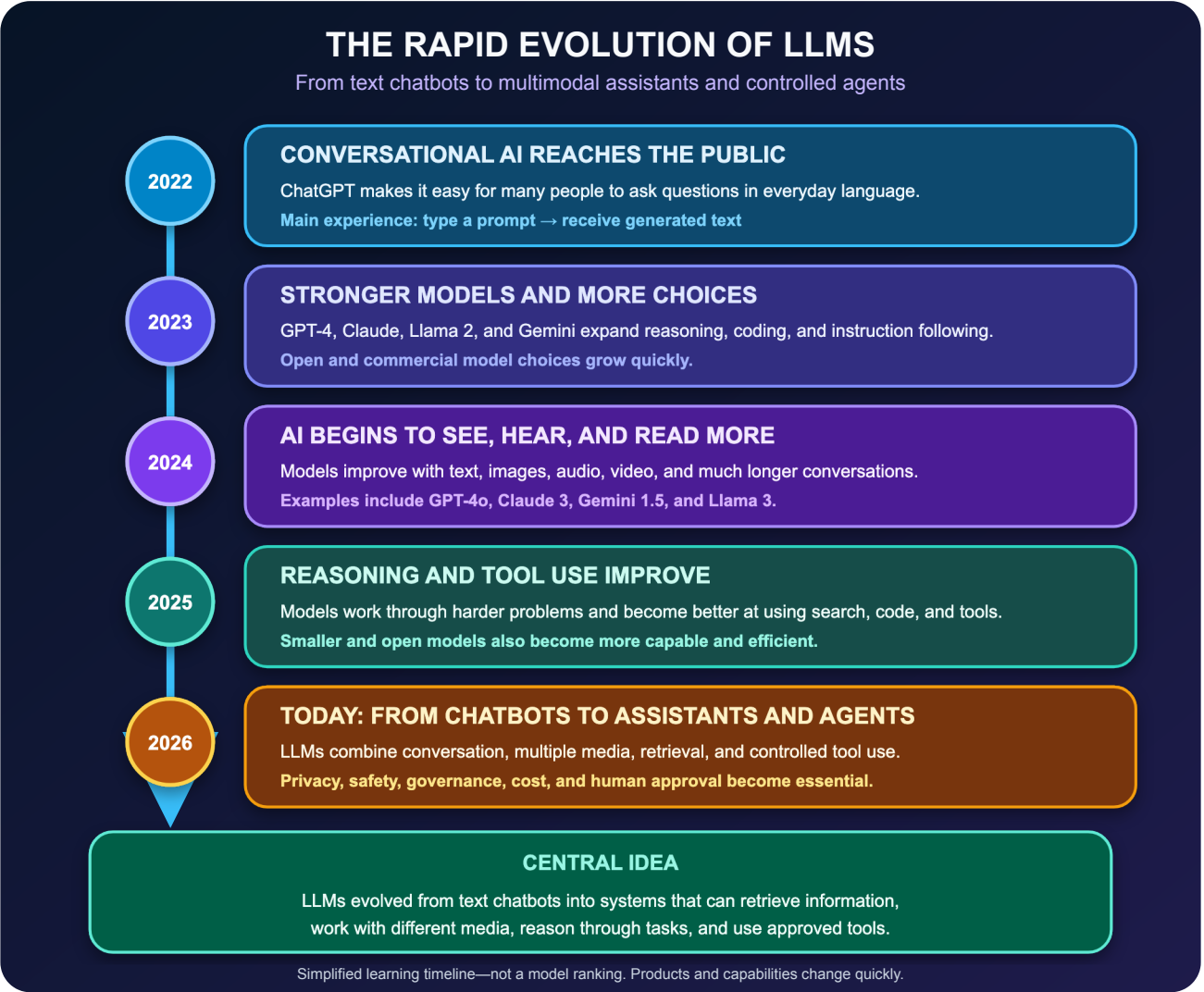
Machine Learning and Generative AI — Same Family, Different Jobs



Plain-language takeaway: Generative AI is not separate from Machine Learning. It is a specialized part of the same AI family, designed to create new content from patterns learned in data.

Healthcare safety: Prediction and generation can both be wrong. Never use these workshop examples for diagnosis, treatment, medication dosing, or real patient-care decisions.

The Rapid Evolution of LLMs — 2022 to Today



Plain-language takeaway: LLMs changed quickly—from answering text questions to working with images, audio, documents, and tools. Human verification remains necessary at every stage.

Timeline note: This is a simplified educational overview through July 2026. Model availability and capabilities vary by provider, product, region, and date.

0. Ground Rules and Helper Functions

- Run cells from top to bottom.
- Change only fictional examples.
- Treat every LLM response as a draft requiring review.
- Follow your organization's privacy, security, and AI policies.

Important mental model: An LLM predicts likely text. It does not know a patient, independently verify facts, or replace clinical judgment.

```

import math
import random
import re
from collections import Counter

def simple_tokenize(text):
    """Split text into simple lowercase word tokens."""
    return re.findall(r"\b\w+\b", text.lower())

def softmax(scores):
    """Turn scores into proportions that add up to 1."""
    shifted = [score - max(scores) for score in scores]
    values = [math.exp(score) for score in shifted]
    total = sum(values)
    return [value / total for value in values]

def bar_chart(label, value, max_value=1.0, width=24):
    """Print a small text bar chart."""
    filled = round((value / max_value) * width) if max_value else 0
    filled = max(0, min(width, filled))
    print(f"{label:<18} |{'#' * filled}{ '-' * (width - filled)}| {value:.3f}")

def bag_of_words(text, vocabulary):
    """Create a simple word-count vector (not a learned embedding)."""
    counts = Counter(simple_tokenize(text))
    return [counts[word] for word in vocabulary]

def cosine_similarity(vector_a, vector_b):
    """Compare two vectors; 1 means same direction, 0 means no overlap."""
    dot = sum(a * b for a, b in zip(vector_a, vector_b))

```

```
length_a = math.sqrt(sum(a * a for a in vector_a))  
length_b = math.sqrt(sum(b * b for b in vector_b))  
return dot / (length_a * length_b) if length_a and length_b else 0.0
```

```
random.seed(7)  
print("Helpers loaded. All examples use fictional data.")
```

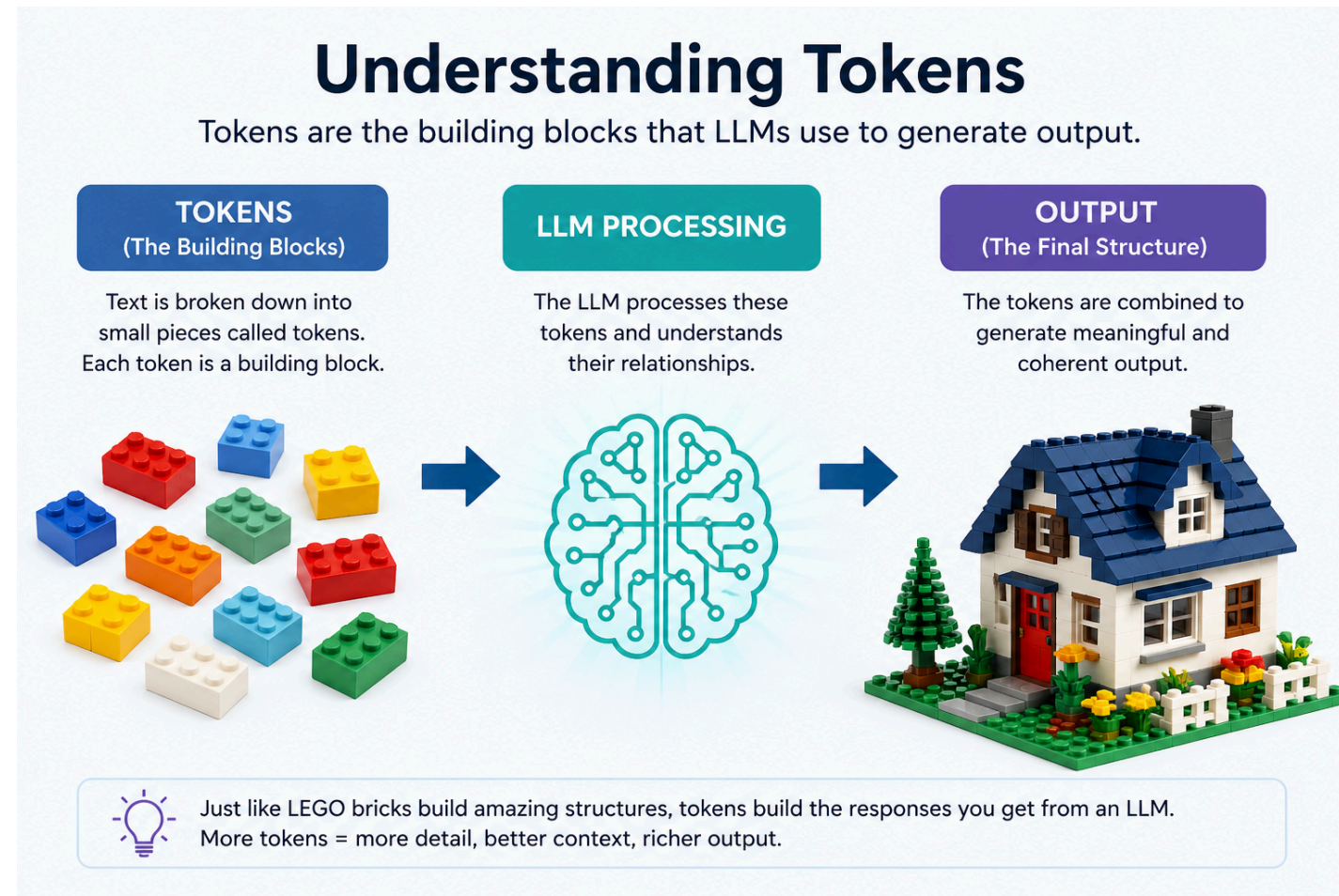
Helpers loaded. All examples use fictional data.

1. Tokens — How an LLM Reads Text

A **token** is a small piece of text. Real tokenizers may use words, punctuation, or parts of words. To keep this workshop simple, we use lowercase words.

The Original LEGO Analogy

The same token idea can also be pictured as LEGO bricks: small pieces are processed and combined into a complete output.



Healthcare connection: A clinical message is also divided into small token pieces before the model processes it. The LEGO picture explains the general concept; the healthcare token picture below applies it to a fictional appointment message.

TOKEN ANALOGY: A MESSAGE BECOMES SMALL LABELED PIECES



FICTIONAL CLINIC MESSAGE

“Your appointment is
Monday at 10 AM.”

One message for a person



The LLM receives separate pieces

your

appointment

is

monday

at

10

am

7 simple tokens



KEY IDEA

The model processes numbered pieces, not the complete message as a clinician sees it.

Healthcare connection: A long clinical note uses many tokens. More tokens require more processing and may exceed a model's working context.

Safety check: Tokenization does not remove identifiers. A patient's name split into tokens is still PHI.

```
message = "Your appointment is Monday at 10 AM."
tokens = simple_tokenize(message)

print("Fictional message:", message)
print("Tokens:", tokens)
print("Token count:", len(tokens))
```

Fictional message: Your appointment is Monday at 10 AM.

Tokens: ['your', 'appointment', 'is', 'monday', 'at', '10', 'am']

Token count: 7

Key Idea

The main point:

An LLM does not process a clinical message as one complete thought. It first divides the text into small numbered pieces called **tokens**. Everything the model reads and writes is built from these pieces.

Healthcare connection: A discharge instruction, appointment message, policy, or fictional clinical note is converted into tokens before the model processes it.

What Counts as One Token?

In a real LLM, a token may be:

- A short common word: the, is, patient
- Part of a longer word: cardio + vascular
- A punctuation mark: ., ,, :
- A number or part of a number: 10, 00
- A symbol or space pattern

For this workshop, the Python examples mostly use whole lowercase words. Real tokenizers are more detailed, so their token counts may differ.

Healthcare example: Appointment: Monday at 10:00 AM may split words, punctuation, and numbers into separate tokens.

Why Not Use Only Whole Words or Individual Letters?

Approach	Example: cardiovascular	Main limitation
Individual characters	c , a , r , d , i , ...	Creates very long sequences and requires more processing
Whole words	cardiovascular	New, rare, misspelled, or specialized terms may not be in the vocabulary
Subwords 	cardio + vascular	Handles unfamiliar terms while keeping sequences reasonably short

Most modern LLMs use subword tokenization. Common words may stay together, while uncommon clinical terms, medication names, abbreviations, or misspellings may be divided into several pieces.

Safety check: Splitting a medical term into tokens does not mean the model understands its clinical meaning.

Surprising Tokenization Facts

- **A person's name may become several tokens.** Tokenization does not anonymize it; the name is still PHI.
 - **Medical terminology may use many tokens.** A rare condition or medication name can be divided into multiple subwords.
 - **Punctuation and spacing matter.** `10:00 AM`, `10 AM`, and `10AM` may produce different tokens.
 - **Abbreviations can be ambiguous.** The same letters may mean different things in different specialties.
 - **Different languages tokenize differently.** The English rule of thumb may not work for multilingual healthcare content.
 - **Long notes consume many tokens.** Repeated headers, copied-forward text, and large policy documents increase the total.
-

Why Does Vocabulary Size Matter?

A model's **vocabulary** is the fixed set of token pieces its tokenizer recognizes. A larger vocabulary can represent more common words and text patterns with fewer pieces, but it also increases the number of possible next-token choices.

For healthcare professionals, this means:

- A familiar word such as `patient` may be one token.
- A specialized term may be split into several tokens.
- A new medication or uncommon name can still be represented by smaller pieces.
- More pieces use more of the model's context window.

Vocabulary size affects efficiency, but it does **not** guarantee medical knowledge, accuracy, or safety.

The Cost Angle in Production AI

Many hosted AI services charge for the number of **input tokens** and **output tokens** processed. A short appointment message costs less to process than a long clinical document.

Healthcare examples that can increase token usage include:

- Long fictional clinical notes
- Multiple policy documents supplied to RAG
- Repeated instructions in a prompt
- Long generated summaries
- Large conversation histories

Reducing unnecessary text may lower cost and improve focus. Never remove clinically important context merely to save tokens, and always follow organizational policy.

Rule of thumb for English text:

1 token $\approx$ 4 characters, or about $\frac{3}{4}$ of a word.

1,000 tokens $\approx$ 750 words, or roughly 1–2 pages of ordinary text.

These are only estimates. Medical terms, numbers, abbreviations, formatting, and different languages can change the count.

Privacy reminder: Token cost never changes the privacy rule—do not enter real PHI into an unapproved AI tool.

1A. Token IDs — Text Becomes Numbers

📌 The most important thing in this entire section:

An AI model NEVER actually sees words. It only ever sees NUMBERS.

Every token receives a unique **token ID** from a lookup table called the **vocabulary**. The model performs mathematics using these numbers—it does not read text the way a healthcare professional does.

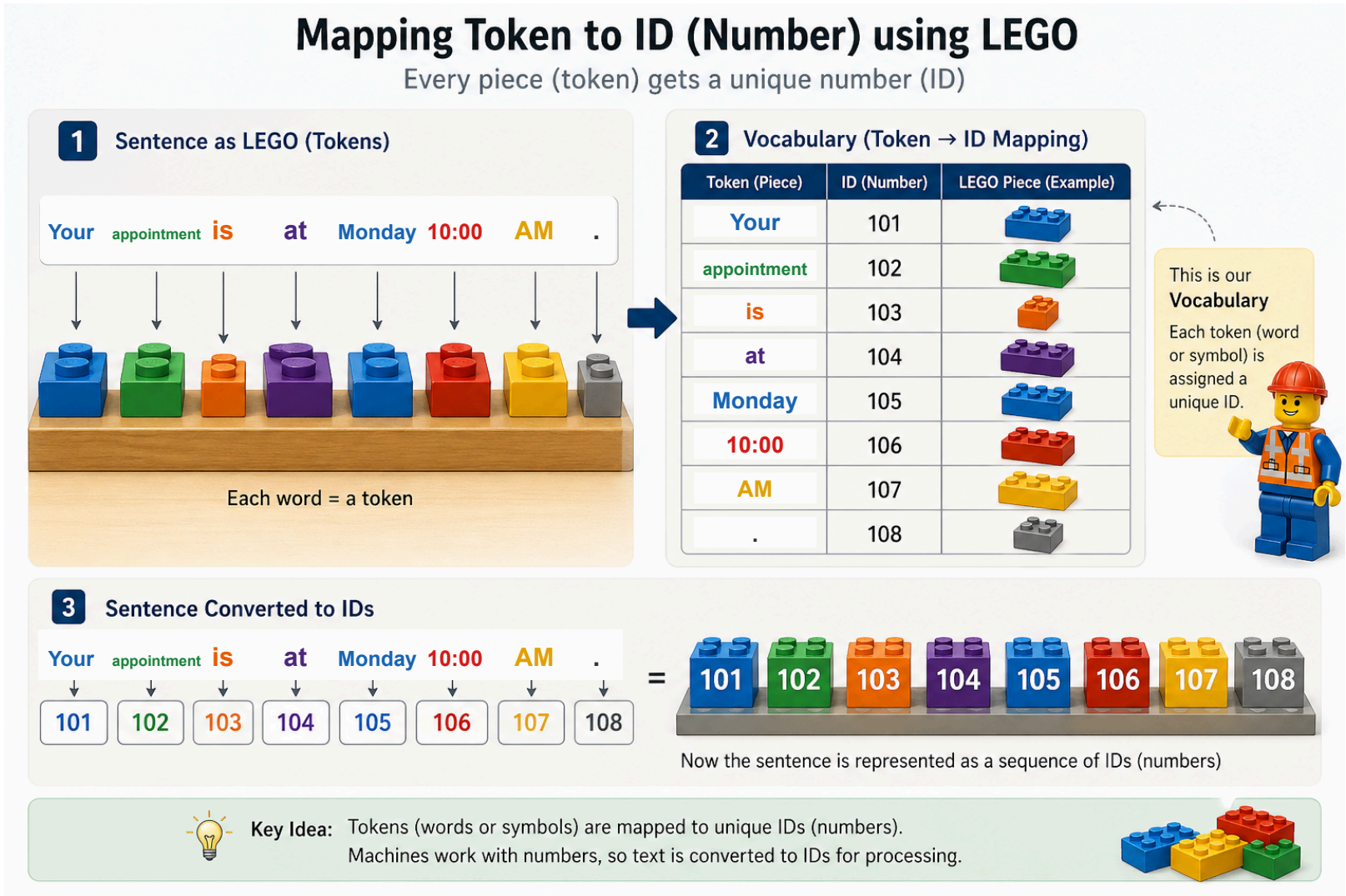
For example:

- A person sees the word `appointment`.
- The tokenizer may convert it to a number such as `102`.
- The model processes `102` and other token IDs through mathematical operations.
- The ID is only a label. It is **not** a clinical score, code, or diagnosis.

Healthcare connection: A fictional appointment message becomes a sequence of token IDs before the model processes it. Tokenization does not remove PHI or make sensitive text anonymous.

LEGO Token-to-ID Picture

The original LEGO token-ID picture is adapted below using the fictional example **Your appointment is at Monday 10:00 AM.**



```
sentence = "Your appointment is at Monday 10:00 AM"

# The picture also shows the final punctuation mark as its own token.
tokens = ["Your", "appointment", "is", "at", "Monday", "10:00", "AM", "."]

vocabulary = {
    "Your": 101,
    "appointment": 102,
    "is": 103,
    "at": 104,
    "Monday": 105,
    "10:00": 106,
    "AM": 107,
    ".": 108,
}

token_ids = [vocabulary[token] for token in tokens]
id_to_token = {number: token for token, number in vocabulary.items()}
decoded_tokens = [id_to_token[number] for number in token_ids]

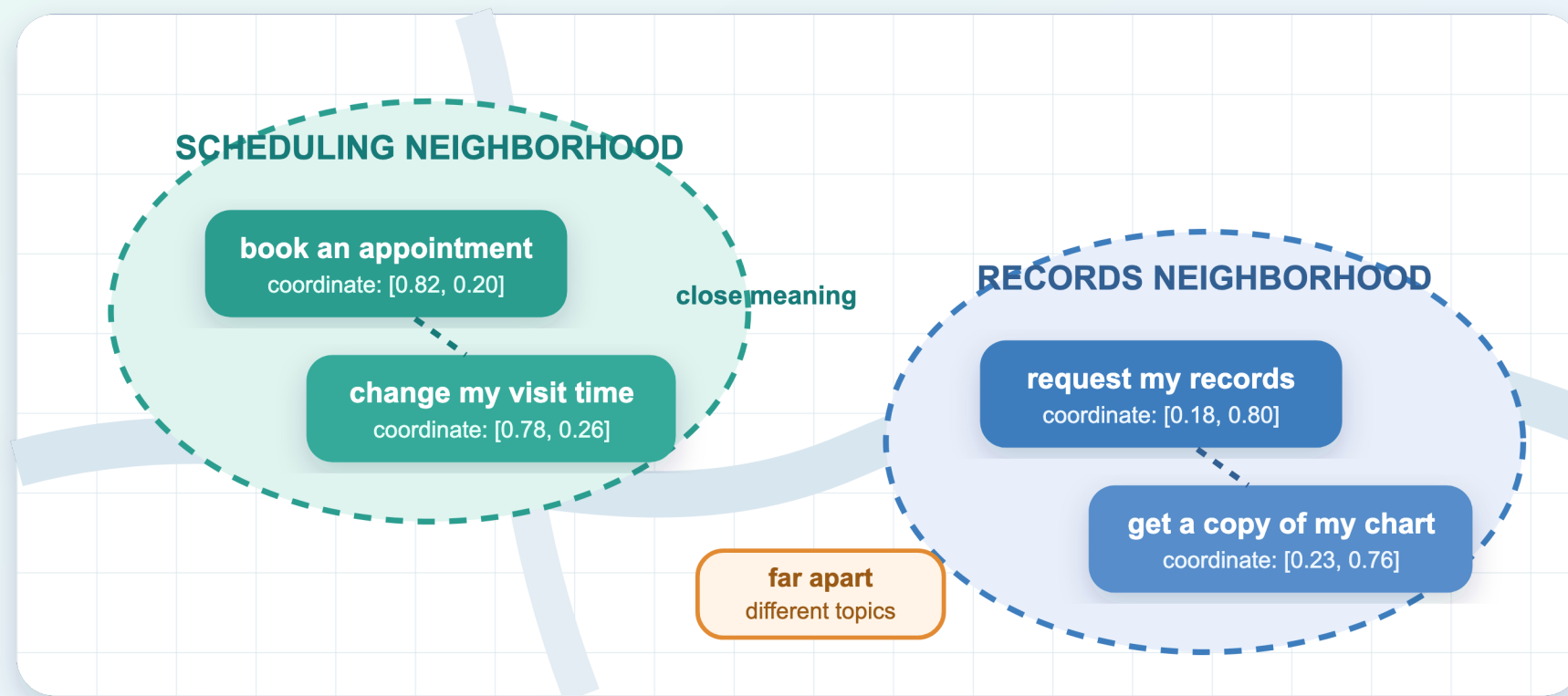
print("Fictional sentence:", sentence)
print("Tokens:", tokens)
print("Token IDs:", token_ids)
print("Decoded again:", decoded_tokens)
```

```
Fictional sentence: Your appointment is at Monday 10:00 AM
Tokens: ['Your', 'appointment', 'is', 'at', 'Monday', '10:00', 'AM', '.']
Token IDs: [101, 102, 103, 104, 105, 106, 107, 108]
Decoded again: ['Your', 'appointment', 'is', 'at', 'Monday', '10:00', 'AM', '.']
```

2. Embeddings — A Numeric Map of Meaning

A learned **embedding** represents text as a list of numbers. Similar meanings tend to be near each other in that numeric space.

EMBEDDING ANALOGY: A MAP WHERE SIMILAR MEANINGS ARE NEIGHBOURS



Real embeddings use hundreds or thousands of coordinates learned from data.

The next code uses **word counts**, not learned embeddings. It is a transparent toy model that helps demonstrate vectors and similarity.

Healthcare connection: Real embeddings can help retrieve relevant approved policies or group administrative messages by meaning.

Safety check: Similarity is not clinical correctness. Humans must check the retrieved source and context.

The City Map Analogy

Think of every word or sentence as having an address on a very large map of meaning:

- `book appointment monday` and `change appointment monday` live in the same **scheduling neighborhood**.
- `request medical records` lives in a different **records neighborhood**.
- Finding similar text is like finding nearby addresses on the map.

A **token ID** is only a label, like a medical record shelf number. An **embedding** is a list of coordinates that describes where the text sits on the meaning map.

Real learned embeddings may use hundreds or thousands of coordinates. The code below intentionally uses a much simpler **word-count vector** so every number is easy to inspect.

How the Current Healthcare Example Works

1. Build one shared vocabulary from the three fictional phrases.
2. Count whether each vocabulary word appears in each phrase.
3. Store those counts in the same order to create comparable vectors.
4. Use cosine similarity to compare the directions of those vectors.

Important limitation: This teaching example mostly measures shared words. A real learned embedding can recognize that different words may express similar meaning.

Healthcare safety: A nearby vector means “similar according to the representation,” not “clinically correct,” “safe,” or “appropriate for the same patient.”

```

phrases = [
    "book appointment monday",
    "change appointment monday",
    "request medical records",
]

phrase_vocabulary = sorted({word for phrase in phrases for word in simple_tokenize(phrase)})
vectors = [bag_of_words(phrase, phrase_vocabulary) for phrase in phrases]

print("Vocabulary:", phrase_vocabulary)
for phrase, vector in zip(phrases, vectors):
    print(f"{phrase!r} -> {vector}")

print("\nSimilarity:")
print("appointment phrases:", round(cosine_similarity(vectors[0], vectors[1]), 3))
print("appointment vs records:", round(cosine_similarity(vectors[0], vectors[2]), 3))

```

```

Vocabulary: ['appointment', 'book', 'change', 'medical', 'monday', 'records', 'request']
'book appointment monday' -> [1, 1, 0, 0, 1, 0, 0]
'change appointment monday' -> [1, 0, 1, 0, 1, 0, 0]
'request medical records' -> [0, 0, 0, 1, 0, 1, 1]

```

```

Similarity:
appointment phrases: 0.667
appointment vs records: 0.0

```

2B. Measuring Similarity with Cosine Similarity

Once we have embeddings (number lists), we can measure how **close** two phrases are.

We use **cosine similarity**:

- Result of **1.0** = identical direction (very similar representation)
- Result of **0.0** = perpendicular (unrelated in this example)
- Result of **-1.0** = opposite direction

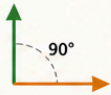
Imagine two arrows on a map—cosine similarity measures the angle between them.

3B. MEASURING SIMILARITY WITH COSINE SIMILARITY

Cosine similarity measures the angle between two embedding vectors (arrows).



COSINE SIMILARITY = 1.0
Identical direction
(Very similar meaning)



COSINE SIMILARITY = 0.0
Perpendicular
(Unrelated)



COSINE SIMILARITY = -1.0
Opposite direction
(Opposite meaning)

OUR THREE SENTENCES

s1: "cat drinks milk"
s2: "cat drinks water"
s3: "sun shines bright"

Vocabulary (shared)

bright [0] cat [1] drinks [2] milk [3] shines [4] sun [5] water [6]

Embeddings (bag-of-words count vectors)

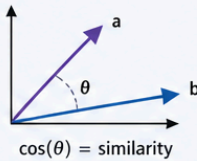
s1: "cat drinks milk"	0	1	1	1	0	0	0
s2: "cat drinks water"	0	1	1	0	0	0	1
s3: "sun shines bright"	1	0	0	0	1	1	0

HOW COSINE SIMILARITY WORKS

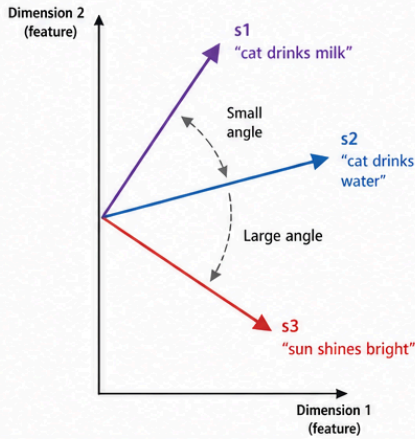
$$\text{Cosine similarity} = \frac{\text{dot}(a, b)}{\|a\| \times \|b\|}$$

- dot(a, b): dot product (overlap between vectors)
- $\|a\|$, $\|b\|$: vector lengths (magnitudes)

Result is between -1.0 and 1.0



VISUALISING OUR THREE SENTENCES



s1 vs s2
Small angle
Cosine similarity ≈ 0.82
(high)

s1 vs s3
Large angle
Cosine similarity ≈ 0.35
(low)

s1 vs (unrelated)
Perpendicular
Cosine similarity = 0.00
(none)

s1 vs s2

DETAILED EXAMPLE CALCULATIONS

s1 vs s3

Step A — Dot product
 $(0 \times 0) + (1 \times 1) + (1 \times 1) + (1 \times 0) + (0 \times 0) + (0 \times 0) + (0 \times 1) = 2$

Step B — Vector lengths
 $\|s1\| = \sqrt{0^2 + 1^2 + 1^2 + 1^2 + 0^2 + 0^2 + 0^2} = \sqrt{3} \approx 1.732$
 $\|s2\| = \sqrt{0^2 + 1^2 + 1^2 + 0^2 + 0^2 + 0^2 + 1^2} = \sqrt{3} \approx 1.732$

Step C — Final similarity
 $2 \div (1.732 \times 1.732) = 2 \div 3 \approx 0.667$

Step A — Dot product
 $(0 \times 1) + (1 \times 0) + (1 \times 0) + (1 \times 0) + (0 \times 1) + (0 \times 1) + (0 \times 0) = 0$

Step B — Vector lengths
 $\|s1\| = \sqrt{3} \approx 1.732$
 $\|s3\| = \sqrt{3} \approx 1.732$

Step C — Final similarity
 $0 \div (1.732 \times 1.732) = 0 \div 3 = 0.000$

SUMMARY: SIMILARITY SCORES



Cosine similarity focuses on direction, not magnitude. It tells us how close the meanings are, regardless of sentence length.

The Big Idea

Real learned embeddings can place phrases with **different words but similar meanings** near one another. This supports:

- **Semantic search** — find approved documents by meaning rather than exact wording
- **RAG retrieval** — select relevant policies or guidance for a prompt
- **Administrative message grouping** — organize similar non-clinical requests

Healthcare safety: A high similarity score is only a retrieval signal. It does not prove that a document is current, applicable, clinically correct, or authorized for the user. Always verify the source and context.

3. Parameters — Where the Model's Knowledge Lives

A **parameter** is a number inside the model that was adjusted during training.

The Radio Analogy

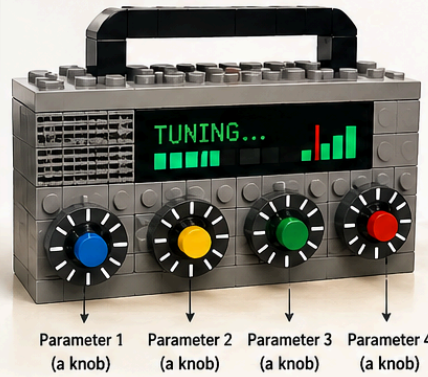
Imagine an old radio with many tuning knobs:

- Each **knob** is a parameter.
- Tuning the radio means adjusting knobs until the signal becomes clearer.
- **Training** adjusts very many parameters to reduce prediction errors.
- After training, the learned parameter values are stored as numbers.

PARAMETERS: The Model's Knobs

A parameter is a number inside the model that was adjusted during training.

THE RADIO ANALOGY



- Each knob is a parameter
- Tune the knobs until you get a clear signal
- Training = turning millions of knobs until the output becomes accurate
- After training, the knobs are locked — stored as numbers in a file

AFTER TRAINING



LOCKED & SAVED

```
model.safetensors  
param_1 = -0.23152  
param_2 = 1.28471  
param_3 = -0.98711  
param_4 = 0.52109  
...  
...  
param_N = -0.00321
```

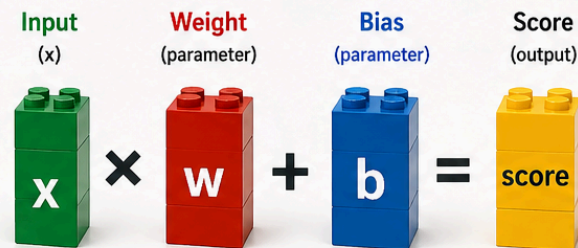
GPT-4 is estimated to have over **1 TRILLION** such knobs (parameters)!

THE SIMPLEST POSSIBLE MODEL

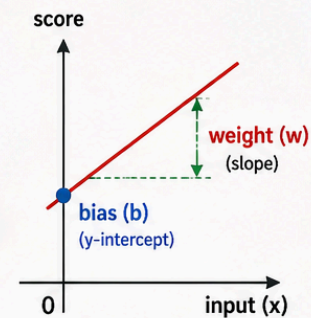
The formula **score = weight × input + bias** has exactly 2 parameters:

weight — how much the input matters (controls the slope)

bias — a baseline offset (controls the starting point)



$$\text{score} = w \times x + b$$



The Simplest Possible Model

The formula `score = weight × input + bias` has exactly two parameters:

- `weight` — how much the input affects the output
- `bias` — a baseline adjustment

This tiny calculation is one building block used inside much larger neural networks.

The Scale of Real Models

Our healthcare teaching model below has only **2 parameters**: a weight and a bias.

Real language models combine millions or billions of learned parameters. The table includes several recent model families and is updated through **July 2026**.

Model	Publicly reported or released parameter scale	What to notice
GPT-3	175 billion	Historical dense-model example
Llama 4 Scout	109 billion total; 17 billion active per token	Mixture-of-experts model
Llama 4 Maverick	400 billion total; 17 billion active per token	Large model, but only part is active for each token
DeepSeek-V3	671 billion total; 37 billion active per token	Mixture-of-experts model
Qwen3-235B-A22B	235 billion total; 22 billion active per token	The name shows total and active scale
GPT-5.6 Sol, Terra, and Luna	Not publicly disclosed	Recent proprietary family; model size is private
Gemini 3.6 Flash	Not publicly disclosed	Recent proprietary efficiency-focused model

Why Do Some Models Have Two Parameter Numbers?

Some newer models use **Mixture of Experts (MoE)**:

- **Total parameters** = all parameters stored in the model.
- **Active parameters** = the smaller subset used to process one token.
- Activating only part of the model can reduce computing cost and response time.

Some companies do not publish parameter counts for proprietary models. **Not publicly disclosed** is more accurate than relying on unofficial estimates.

Model names and availability change quickly. Always check the provider's current documentation before making architecture, capability, or cost decisions.

More parameters can represent more language patterns, but parameter count alone does not determine quality, accuracy, speed, cost, or safety.

Healthcare connection: Parameters may encode patterns learned from training data, including errors or bias. They are not a verified clinical database and must not replace approved sources or professional judgment.

Safety check: A confident output can still be incomplete, outdated, or incorrect. Verify consequential healthcare information against current approved sources.

```
def tiny_model(input_value, weight, bias):  
    """A tiny two-parameter teaching model."""  
    return weight * input_value + bias  
  
# Fictional administrative example: estimate forms processed.  
forms_received = 4  
weight = 2.0  
bias = 1.0  
prediction = tiny_model(forms_received, weight, bias)  
  
print("Input:", forms_received, "fictional form batches")  
print("Parameters: weight =", weight, ", bias =", bias)  
print("Toy output:", prediction)  
print("This is arithmetic for learning—not a clinical prediction.")
```

Input: 4 fictional form batches

Parameters: weight = 2.0 , bias = 1.0

Toy output: 9.0

This is arithmetic for learning—not a clinical prediction.

Parameters Control the Output

The input can stay exactly the same while different parameter values produce different outputs. During training, the model searches for parameter values that reduce its errors.

The next example keeps the fictional input at **4 form batches** and changes only the `weight` and `bias`.

Healthcare safety: This is a simple arithmetic demonstration, not a staffing forecast, clinical prediction, or operational recommendation.

```

# Keep the fictional administrative input the same.
form_batches = 4

# Change only the model's two parameters.
parameter_examples = [
    (0.5, 0.0),
    (1.0, 0.0),
    (2.0, 1.0),
    (3.0, -2.0),
]

print(f"Same input every time: {form_batches} fictional form batches\n")
print(f"{'Weight':>8} {'Bias':>8} {'Toy output':>12}")
print("-" * 32)

for example_weight, example_bias in parameter_examples:
    toy_output = tiny_model(form_batches, example_weight, example_bias)
    print(f"{example_weight:>8.1f} {example_bias:>8.1f} {toy_output:>12.1f}")

print("\nKey lesson: the input did not change, but the output changed")
print("because the parameter values changed.")
print("Educational arithmetic only—not a healthcare prediction.")

```

Same input every time: 4 fictional form batches

Weight	Bias	Toy output
0.5	0.0	2.0
1.0	0.0	4.0
2.0	1.0	9.0
3.0	-2.0	10.0

Key lesson: the input did not change, but the output changed
because the parameter values changed.
Educational arithmetic only—not a healthcare prediction.

4. Attention — Which Words Matter in This Context?

This is the breakthrough idea behind the "Transformer" architecture that powers all modern LLMs.

Landmark AI Paper

Attention Is All You Need (2017)

One-line fact: This paper introduced the Transformer architecture that became the technical foundation for modern LLMs and today's multibillion-dollar generative-AI industry.

Why This Paper Was a Big Deal

Imagine trying to understand a book by reading **one word at a time from beginning to end** and carrying everything in your memory. Older AI systems worked more like this, which made learning from long passages slower and more difficult.

The Transformer introduced a better approach. It is like quickly looking across a page, noticing the **important words and connections**, and using them to understand the story.

For example, if a page repeatedly mentions **hospital, doctor, appointment**, and **patient**, the AI learns that these words are connected and probably important to the meaning of that passage.

This helped AI:

- Examine many words together instead of only one after another.
- Focus on the words and connections that matter most.
- Understand larger amounts of text more efficiently.
- Train faster by doing more work at the same time on powerful computers.

💡 **Central idea:** This paper helped AI move from reading language mostly one step at a time to examining many words together, which helped make today's LLMs possible.

Small accuracy note: An LLM does not literally scan an unlimited whole book at once. It examines the text that fits inside its available context window.

The Doctor Analogy

Imagine a doctor reviewing a patient's medical file before a consultation.

- The file contains hundreds of data points — age, weight, shoe size, favourite colour, blood pressure, cholesterol, family history, sleep patterns...
- The doctor does **not** read every line with equal care
- When checking for heart disease, they immediately **spotlight** "cholesterol", "blood pressure", and "family history"
- They mostly **ignore** "shoe size" or "favourite colour" — those are irrelevant to the question at hand

Attention lets a model do exactly the same thing:

- For each word it is processing, it asks: *"which other words in this sentence are most relevant to me right now?"*
- It assigns high **attention weights** to relevant words and low weights to irrelevant ones
- It then builds its understanding by combining the weighted information

ATTENTION — HOW AI DECIDES WHAT TO FOCUS ON

Focus on what matters. Ignore the rest.

THE DOCTOR ANALOGY

A doctor reviews a patient's file to check for heart disease.



Age: 52
Weight: 82 kg
Shoe size: 9
Favourite colour: Blue
Blood pressure: 160/95
Cholesterol: 240 mg/dL
Eye colour: Brown
Family history: Heart disease
Sleep: 5–6 hours
Mobile phone: iPhone
... hundreds more lines ...

Somewhat relevant

Highly relevant (spotlight)

Mostly ignored



The doctor **spotlights** what matters for the question (**heart disease**) and mostly ignores the rest.

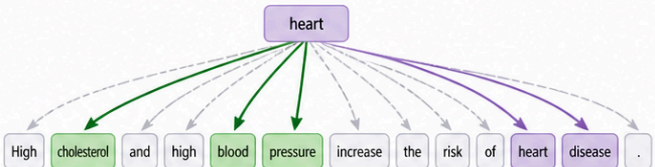
THE AI / TRANSFORMER

For each word, the model decides which other words in the sentence are most relevant right now.

Example sentence:

High **cholesterol** and high **blood pressure** increase the risk of **heart disease**.

When processing the word “heart” ...



→ High attention (very relevant)
--> Low attention (less relevant)
→ Strong attention (very relevant)



The model assigns high attention weights to relevant words and **low** weights to irrelevant ones.

ATTENTION WEIGHTS (EXAMPLE)

Words	High	cholesterol	and	high	blood	pressure	increase	the	risk	of	heart	disease	.
Attention weight (from “heart”)	0.02	0.18	0.01	0.02	0.17	0.16	0.02	0.01	0.02	0.01	0.22	0.16	0.00

→ Attention lets the model focus on the most relevant information — just like a doctor does.

Why Attention Solves a Hard Problem

A word can mean different things in different sentences. Consider the word "**discharge**":

- *"The patient was ready for **discharge** from the hospital."* Here, discharge means leaving the hospital.
- *"The nurse documented **discharge** from the wound."* Here, discharge means fluid from the wound.

The word is spelled exactly the same way, but its meaning changes.

Without context, the model only sees the word `discharge` . With attention, it looks at the surrounding words:

- In the first sentence, `discharge` focuses strongly on `ready` and `hospital`.
- In the second sentence, `discharge` shifts its focus to `documented` and `wound`.

Think of attention as a set of **highlighter pens**. For every word, the model highlights the other words that are most useful at that moment.

Important: The scores below are hand-written for teaching. A real Transformer learns attention patterns during training.

```

# ATTENTION: Which words help explain the target word?
#
# A real Transformer calculates attention scores automatically.
# Here, we write the scores ourselves so the idea is easy to see.

def show_attention(target_word, sentence_words, raw_relevance_scores):
    """Show the attention paid to each word while processing target_word."""
    attention_weights = softmax(raw_relevance_scores)
    max_weight = max(attention_weights)

    print(f"How much attention does '{target_word}' pay to each word?")
    print("Longer bar = more influence on the target word in this example.\n")

    for word, weight in zip(sentence_words, attention_weights):
        bar_chart(word, weight, max_value=max_weight)

    print()
    return attention_weights

# CONTEXT 1: "discharge" means leaving the hospital.
hospital_words = [
    "the", "patient", "was", "ready", "for",
    "discharge", "from", "the", "hospital",
]
hospital_scores = [0.1, 1.2, 0.2, 2.7, 0.1, 2.0, 0.2, 0.1, 3.3]

print("=" * 65)
print("SENTENCE 1: 'The patient was ready for discharge from the hospital.'")
print("=" * 65)
hospital_attention = show_attention(
    "discharge", hospital_words, hospital_scores
)
print("The strongest clues are 'hospital' and 'ready'.")
print("So 'discharge' means leaving the hospital.\n")

```

```
# CONTEXT 2: The same word now means fluid from a wound.
wound_words = [
    "the", "nurse", "documented", "discharge",
    "from", "the", "wound",
]
wound_scores = [0.1, 1.0, 2.0, 2.0, 0.1, 0.1, 3.5]

print("=" * 65)
print("SENTENCE 2: 'The nurse documented discharge from the wound.'")
print("=" * 65)
wound_attention = show_attention(
    "discharge", wound_words, wound_scores
)
print("The strongest clues are 'wound' and 'documented'.")
print("So 'discharge' means fluid from the wound.\n")

print("KEY INSIGHT:")
print("The word did not change. The surrounding context changed.")
print("Attention helps the model focus on the clues that change the meaning.")
print("These teaching scores do not represent clinical reasoning or causation.")
```

=====

SENTENCE 1: 'The patient was ready for discharge from the hospital.'

=====

How much attention does 'discharge' pay to each word?
Longer bar = more influence on the target word in this example.

the	#-----	0.019
patient	###-----	0.057
was	#-----	0.021
ready	#####-----	0.255
for	#-----	0.019
discharge	#####-----	0.126
from	#-----	0.021

the	#-----	0.019
hospital	#####	0.464

The strongest clues are 'hospital' and 'ready'.
So 'discharge' means leaving the hospital.

=====

SENTENCE 2: 'The nurse documented discharge from the wound.'

=====

How much attention does 'discharge' pay to each word?
Longer bar = more influence on the target word in this example.

the	#-----	0.020
nurse	##-----	0.050
documented	#####-----	0.137
discharge	#####-----	0.137
from	#-----	0.020
the	#-----	0.020
wound	#####	0.614

The strongest clues are 'wound' and 'documented'.
So 'discharge' means fluid from the wound.

KEY INSIGHT:
The word did not change. The surrounding context changed.
Attention helps the model focus on the clues that change the meaning.
These teaching scores do not represent clinical reasoning or causation.

Attention: How AI Decides What to Focus On

The code above uses the same word, `discharge`, in two contexts. The bars make the model's changing spotlight visible.

4. ATTENTION — HOW AI DECIDES WHAT TO FOCUS ON



For each word the model processes, it looks at all other words in the sentence and asks:
“Which words are most relevant to me right now?”

- High attention = more influence
- Low attention = less influence

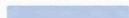
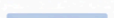


Why attention solves a hard problem: The word “bank” can mean different things.
Attention lets the model look at the right context words to understand the correct meaning.

SENTENCE 1 (FINANCIAL)

“I deposited money at the **bank**.”

 When processing “**bank**”, how much attention does it pay to each word?

Word	Raw relevance score (set by the model)	Attention weight (after softmax) adds up to 100%
I	0.2 	3.1% 
deposited	2.5 	19.1% 
money	3.0 	31.6% 
at	0.1 	2.8% 
the	0.1 	2.8% 
bank	2.0 	20.6% 

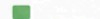
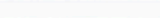
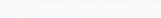
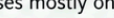
“bank” focuses mostly on “**money**” and “**deposited**”

→ Understands: **financial institution**

SENTENCE 2 (GEOGRAPHICAL)

“We had a picnic on the river **bank**.”

 When processing “**bank**”, how much attention does it pay to each word?

Word	Raw relevance score (set by the model)	Attention weight (after softmax) adds up to 100%
We	0.1 	1.3% 
had	0.1 	1.3% 
a	0.1 	1.3% 
picnic	2.8 	22.5% 
on	0.2 	1.8% 
the	0.1 	1.3% 
river	3.5 	36.6% 
bank	2.0 	14.0% 

“bank” focuses mostly on “**river**” and “**picnic**”

→ Understands: **river edge / land by the water**



KEY INSIGHT:
The SAME word “bank” attends to completely different words depending on the sentence.



This is how attention gives words different meanings in different contexts — automatically.



What the Demonstration Shows

- In the hospital context, **discharge** gives the most attention to **hospital** and **ready**.
- In the wound context, **discharge** shifts its attention to **wound** and **documented**.
- The target word stays the same. The important surrounding words change.
- Softmax converts the hand-written scores into percentages that add up to 100%.

One-Sentence Takeaway

Attention helps each word look at the other words that matter most for understanding its current context.

Healthcare safety: Attention weights show internal language relationships. They do not provide a diagnosis, clinical reasoning, or proof that one event caused another.

5. Next-Token Prediction — How Text Is Generated

An LLM repeatedly predicts a likely next token.

5. NEXT-TOKEN PREDICTION — HOW AI GENERATES TEXT



A language model does one simple thing:
It predicts the most likely next token.
Then it appends that token and repeats.

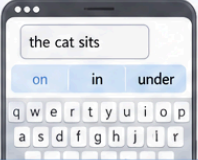
Input so far
the cat sits

Model predicts next token


Append it
the cat sits on

Predict again
the cat sits on the ...

THE AUTOCOMPLETE ANALOGY



- Your phone learns from your typing.
- An LLM learns from billions of web pages, books, and code.

5A. BUILD A TINY PREDICTION MODEL FROM TEXT

Training text (our small corpus)

- the cat sits on the mat
- the cat eats fish
- the dog sits on the rug
- the dog eats food
- the cat sits by the dog
- the cat plays with the dog

Step 1: Tokenize

the cat sits on the mat the cat eats fish the dog ...
(Convert text → tokens)

Step 2: Count what comes next

For every word, count which word follows it.

After 'cat'	After 'the'	After 'sits'	After 'eats'	After 'dog'
sits: 2	cat: 3	on: 2	fish: 1	sits: 1
eats: 1	dog: 2	by: 1	food: 1	eats: 1
plays: 1				plays: 1

→ Most likely next word (top prediction)

After 'the' →	cat	After 'sits' →	on
After 'cat' →	sits	After 'eats' →	fish
After 'dog' →	sits	After 'plays' →	with

★ These counts are learned patterns — just like the parameters in a real LLM.

Step 3: Generate text (always pick / the most probable next word)

Start with a word, repeatedly predict the next.

Start = 'the' → the → cat → sits → on → the → cat → sits → on → the → ...
(loops)

Start = 'cat' → cat → sits → on → the → cat → sits → on → the → cat → ...
(loops)

Start = 'dog' → dog → sits → on → the → cat → sits → on → the → cat → ...
(loops)

WHY THIS MATTERS



A real LLM uses embeddings, attention, and billions of parameters to compute these probabilities — for every possible next token.

→ It picks the most likely (or samples from the top few) and keeps going, one token at a time.

→ That's how it writes essays, answers questions, codes programs, and holds conversations! 

NOTE: We used “greedy” decoding (always pick the top token). Real models use smarter strategies (sampling, temperature, top-k, nucleus sampling, etc.) for more natural and diverse output.

The model follows a short loop:

1. Read all tokens already written.
2. Calculate probabilities for possible next tokens.
3. Select one token.
4. Append that token to the text.
5. Repeat the process to generate the next token.

This is powerful autocomplete, not independent clinical understanding. A fluent sentence can still be wrong.

Safety check: Never treat confidence of wording as confidence of medical evidence. Generated appointment details still need an approved source and human verification.

```
from collections import defaultdict

training_text = """
your appointment is monday
your appointment is confirmed
your appointment is monday
your visit is confirmed
"""

next_token_counts = defaultdict(Counter)
training_words = simple_tokenize(training_text)
for current_word, next_word in zip(training_words, training_words[1:]):
    next_token_counts[current_word][next_word] += 1

print("Words seen after 'appointment':")
for word, count in next_token_counts["appointment"].most_common():
    print(f"  {word}: {count}")

predicted = next_token_counts["appointment"].most_common(1)[0][0]
print("\nToy prediction: 'appointment' ->", predicted)
```

Words seen after 'appointment':
is: 3

Toy prediction: 'appointment' -> is

6. Training and Loss — Learning from Examples

During training, a model:

1. Makes a prediction.
2. Measures the error, called **loss**.
3. Adjusts parameters slightly.
4. Repeats this many times.

The toy example below learns a simple administrative pattern: $y = 2x + 1$.

Healthcare connection: Training data quality matters. Missing groups, historical inequities, and labeling choices can create biased behavior.

■ **Safety check:** Good average performance does not guarantee safe performance for every population or situation.

```

# Fictional administrative counts only: no patient-care prediction.
training_examples = [(1, 3), (2, 5), (3, 7), (4, 9)]
weight, bias = 0.5, 0.0
learning_rate = 0.02

for step in range(301):
    weight_gradient = 0.0
    bias_gradient = 0.0
    for input_value, correct_output in training_examples:
        error = tiny_model(input_value, weight, bias) - correct_output
        weight_gradient += 2 * error * input_value
        bias_gradient += 2 * error

    count = len(training_examples)
    weight -= learning_rate * weight_gradient / count
    bias -= learning_rate * bias_gradient / count

    if step in (0, 50, 100, 200, 300):
        loss = sum(
            (tiny_model(x, weight, bias) - y) ** 2
            for x, y in training_examples
        ) / count
        print(f"step={step:3}  weight={weight:.3f}  bias={bias:.3f}  loss={loss:.5f}")

```

```

step= 0  weight=1.050  bias=0.190  loss=11.27235
step= 50  weight=2.110  bias=0.677  loss=0.01745
step=100  weight=2.081  bias=0.761  loss=0.00957
step=200  weight=2.045  bias=0.869  loss=0.00288
step=300  weight=2.024  bias=0.928  loss=0.00087

```

7. RAG — Retrieval-Augmented Generation

A trained LLM knows only what it saw during training. It cannot know:

- What happened last week
- Your company's internal documents
- Your personal notes

RAG solves this by connecting the model to an external knowledge base at query time.

Why RAG Is Needed — AI Hallucinations

Imagine asking someone a question they do not know. Instead of saying, “I don’t know,” they make a confident guess that sounds believable.

An LLM can do something similar.

Simple Example

Question:

“What time does Green Valley Clinic close on Fridays?”

The AI has no approved information about this fictional clinic, but it answers:

“Green Valley Clinic closes at 8:00 PM on Fridays.”

The answer sounds clear and professional—but the AI **made up the closing time**.

This is called a **hallucination**: an AI response that sounds believable but is incorrect, invented, outdated, or unsupported by a reliable source.

 **Central idea:** An LLM predicts words that sound likely. It does not automatically check whether those words are true.

How RAG Helps

RAG gives the AI relevant, approved information **before** it creates an answer.

For the clinic-hours question, RAG could retrieve the current approved schedule. The AI can then answer from that schedule instead of guessing.

RAG can help by:

- Providing current and approved information.
- Asking the AI to answer only from retrieved sources.
- Including citations that people can verify.
- Allowing the AI to say, “I don’t know based on the approved sources.”

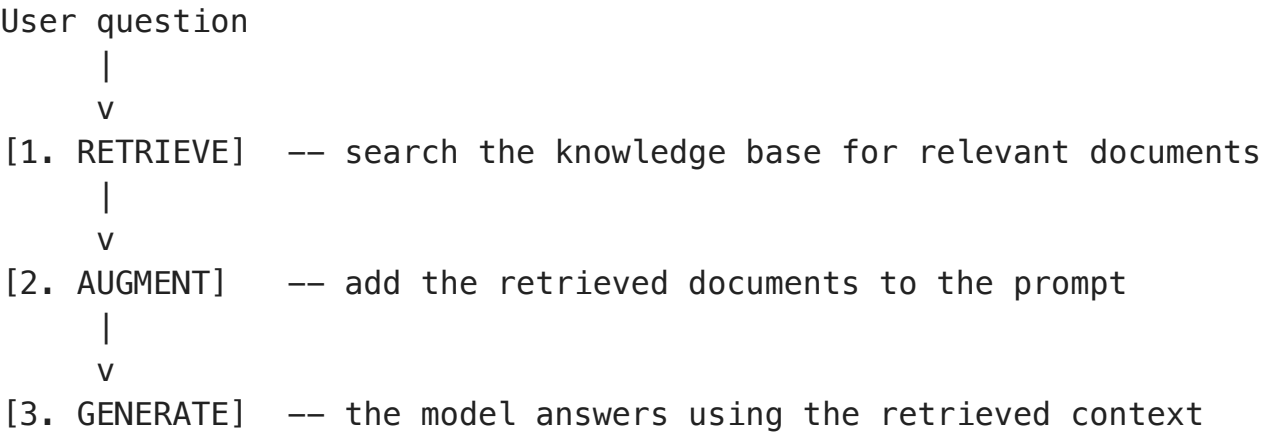
Important: RAG reduces hallucination risk, but it does not remove it completely. The system might retrieve the wrong document or describe it incorrectly. A qualified person must still verify important answers.

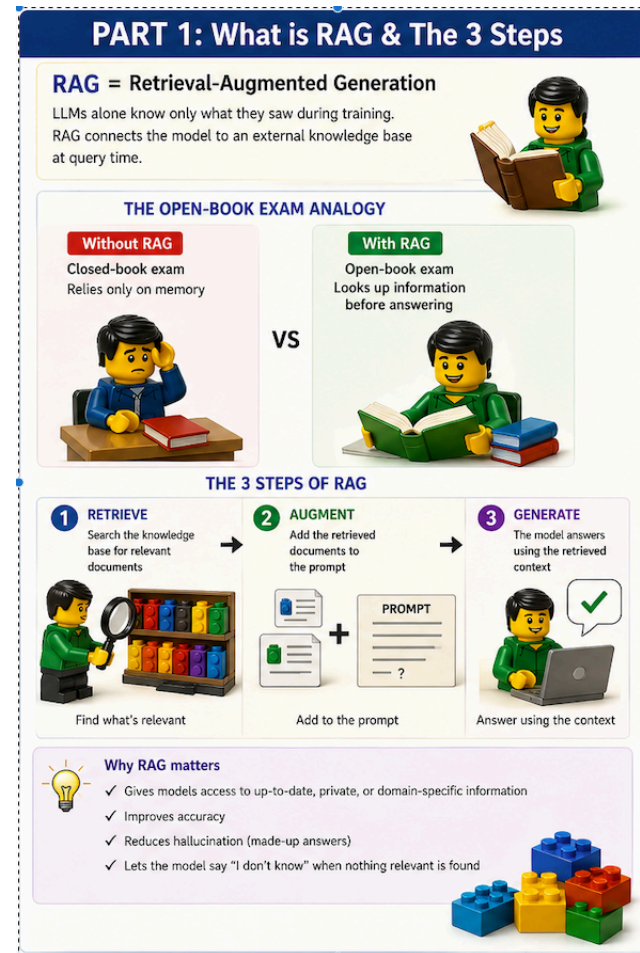
Terminology note: “Hallucination” is an AI term for unsupported or incorrect generated information. It does not mean that the AI thinks, sees, or deliberately lies.

The Open-Book Exam Analogy

- A model without RAG = a student doing a **closed-book exam** — relies only on memory
- A model with RAG = a student doing an **open-book exam** — can look things up before answering

The Three Steps of RAG





Healthcare safety: RAG can improve grounding and traceability, but it does not guarantee correctness. Healthcare implementations still need approved current sources, privacy controls, citations, auditability, and human review.

```

approved_sources = [
    ("POLICY-APPT-01", "Appointment changes may be requested by phone during clinic hours."),
    ("POLICY-RECORDS-02", "Records requests are sent to the Health Information Management team."),
    ("POLICY-BILLING-03", "Billing questions are routed to Patient Financial Services."),
]

rag_stop_words = {"a", "an", "the", "is", "are", "to", "of", "for", "by", "what", "where", "how"}

def retrieve_source(question):
    """Return the approved source with the most meaningful word overlap."""
    question_words = set(simple_tokenize(question)) - rag_stop_words
    scored = []
    for source_id, text in approved_sources:
        source_words = set(simple_tokenize(text)) - rag_stop_words
        scored.append((len(question_words & source_words), source_id, text))
    best_score, source_id, text = max(scored)
    return (source_id, text) if best_score > 0 else None

def grounded_answer(question):
    """Simulate a cautious RAG answer with a source label."""
    match = retrieve_source(question)
    if match is None:
        return "I do not know based on the approved sources."
    source_id, text = match
    return f"Based on [{source_id}]: {text}"

questions = [
    "Where do records requests go?",
    "What is the visitor parking fee?",
]
for question in questions:
    print("Question:", question)

```

```
print("Answer: ", grounded_answer(question))  
print()
```

Question: Where do records requests go?

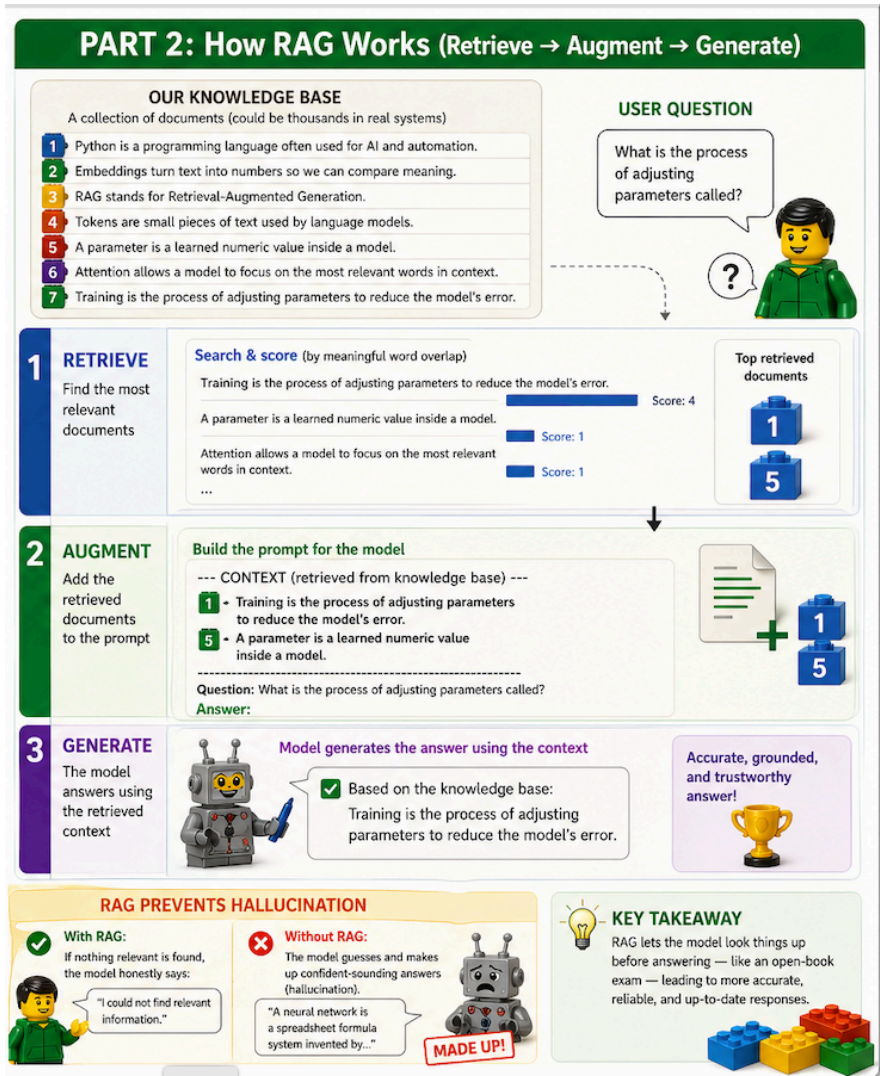
Answer: Based on [POLICY-RECORDS-02]: Records requests are sent to the Health Information Management team.

Question: What is the visitor parking fee?

Answer: I do not know based on the approved sources.

RAG Demonstrated by the Healthcare Code

The picture below follows the exact behavior of the code above.



What Happens for the Records Question

1. **Retrieve:** The question `Where do records requests go?` is compared with the three approved sources.
2. **Augment:** `POLICY-RECORDS-02` is the best match, so its text is added as approved context.
3. **Generate:** The answer repeats the approved information and includes the source label.

What Happens When No Source Matches

For `What is the visitor parking fee?`, the code finds no meaningful overlap. It returns:

I do not know based on the approved sources.

This fallback is important. A grounded system should admit when its approved sources do not contain an answer instead of inventing one.

Healthcare safety: Retrieval can still select an outdated or incorrect source. Generation can still misstate the retrieved text. Use access controls, current approved content, citations, audit logs, and human review.

Why RAG Matters in Practice

Without RAG	With RAG
Model knows only training data	Model can use fresh, private documents
May hallucinate outdated facts	Answers are grounded in retrieved sources
Cannot reference your data	Can reference your internal knowledge base
Hard to audit	You can point to the source document

RAG is one of the most widely deployed GenAI patterns in enterprise applications today.

8. Agents, MCP, and A2A — From One AI Assistant to a Controlled Team

A normal chatbot mainly answers questions:

You ask → AI answers

An **agent** can go one step further. It can examine a task, choose an approved action, use an allowed tool, and return a result for review.

💡 **Central idea:** RAG gives AI approved information. MCP connects an agent to approved tools. A2A allows approved agents to work together. Humans remain responsible for the final decision.

The Healthcare Team Analogy

Imagine a fictional healthcare administrative coordinator named Maya. She receives this request:

“Using fictional information only, look up where records requests go, estimate 25 + 15 minutes, and propose the correct administrative route.”

Maya cannot complete every part alone, so she works with a small team:

- **Policy Agent** — looks up an approved policy.
- **Wait-Time Agent** — performs the simple calculation.
- **Routing Agent** — proposes an administrative destination.
- **Coordinator Agent** — sends work to the specialists and combines their results.
- **Human Reviewer** — checks and approves the final response.

This is a **multi-agent system**: several specialized agents cooperate on one larger task.

RAG, MCP, and A2A in Plain Language

Technology	Simple analogy	What it does
RAG	Approved library	Gives an agent relevant, approved information
MCP	Labeled toolbox	Describes available tools and how to use them
A2A	Secure team handoff	Allows one agent to send work to another agent
Human review	Responsible supervisor	Verifies, approves, corrects, or stops the result

RAG — “Where can I find trusted information?”

The Policy Agent uses RAG to retrieve the approved records policy instead of guessing.

MCP — “Which approved tool can I use?”

MCP describes tools such as `lookup_approved_policy`, `estimate_clinic_wait`, and `propose_admin_route`. It tells an agent what each tool does, what input it expects, and its safety limits.

A2A — “Which other agent can help me?”

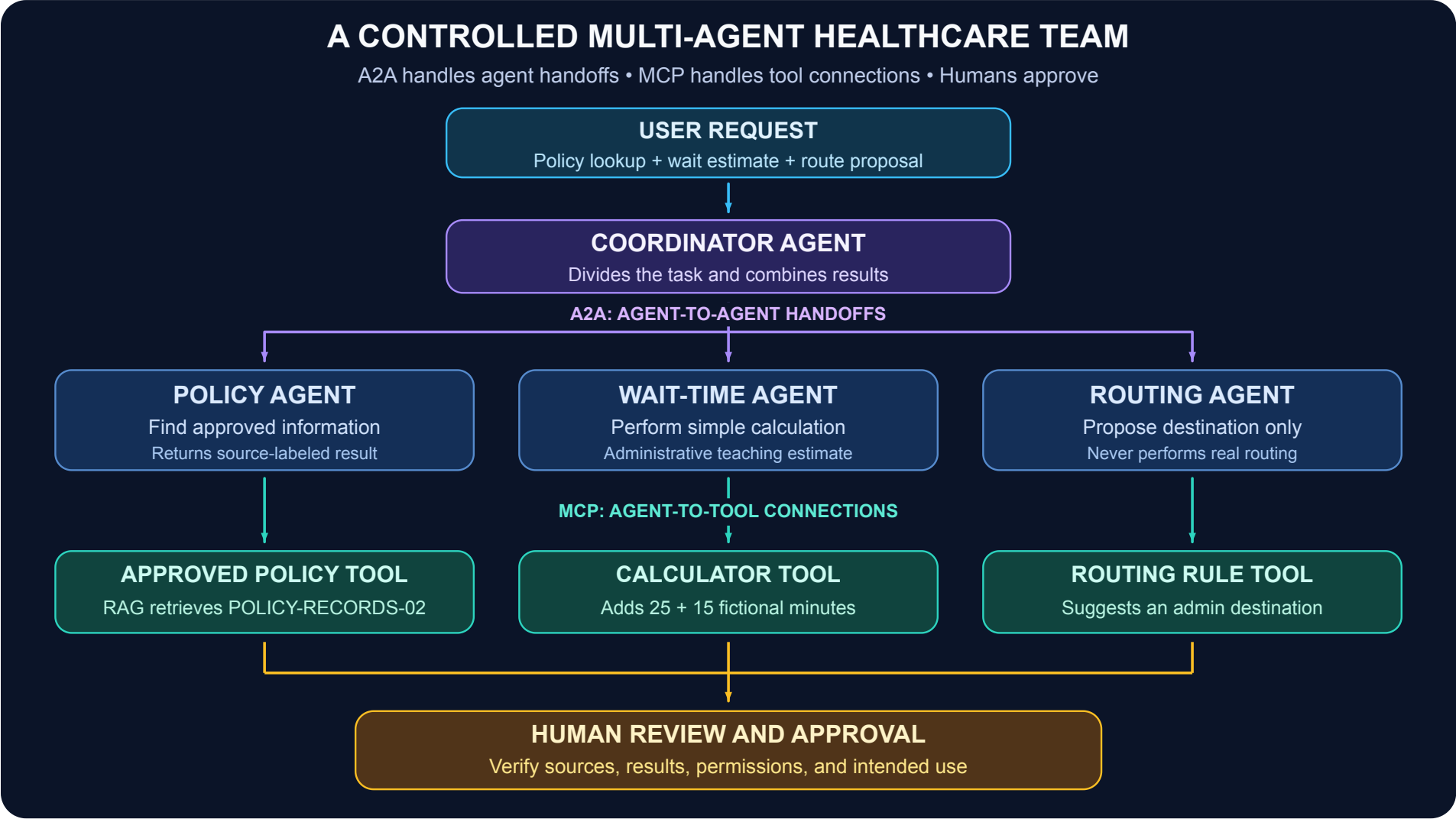
A2A means Agent-to-Agent. It gives agents a standard way to:

1. Introduce their capabilities.
2. Receive a task from another agent.
3. Report progress or ask for clarification.
4. Return a result for review.

Think of A2A as a controlled referral or handoff between members of a healthcare administrative team.

Important distinction: MCP connects an agent to a **tool**. A2A connects an agent to **another agent**.

How the Controlled Team Works



Improved Classroom Demonstration

The next code cell uses:

1. **Three specialist Agent Cards** — each clearly states one capability.
2. **An A2A handoff function** — the Coordinator delegates work to the right specialist.
3. **An MCP tool registry** — each specialist uses only its approved tool.
4. **A structured request** — all three parts are completed instead of selecting only one.
5. **A safe stop** — unknown work is sent to human review rather than an unrelated fallback.

Safety Boundaries

- Share only the minimum information needed between agents.
- Never send PHI to an unauthorized agent or tool.
- Give each agent only the permissions required for its job.
- Record tool calls, agent handoffs, sources, and results.
- Require human approval before routing or operational action.
- Treat every agent response as a draft until verified.

Final takeaway: A multi-agent system should work like a controlled team—not a group of unsupervised bots. Every agent needs a clear job, limited permissions, approved tools, traceable handoffs, and human oversight.

```

# =====
# SECTION 9 – AGENTS, MCP, AND A2A
# =====
# This is a transparent classroom simulation:
#   A2A = agent-to-agent handoff
#   MCP = agent-to-approved-tool connection
# All information and actions below are fictional.
# =====

# ---- Step 1: Approved Administrative Tools ----

def lookup_approved_policy(question):
    """Use the fictional RAG library created in Section 7."""
    return grounded_answer(question)

def estimate_clinic_wait(current_wait, added_minutes):
    """Add two fictional administrative wait-time values."""
    if current_wait < 0 or added_minutes < 0:
        return "Wait-time values must be zero or greater."
    return f"{current_wait + added_minutes} fictional minutes"

def propose_admin_route(request_type):
    """Propose a destination without performing a real action."""
    routes = {
        "records": "Health Information Management",
        "billing": "Patient Financial Services",
        "appointment": "Scheduling",
    }
    destination = routes.get(request_type.lower())
    if destination is None:
        return "No approved route found. Send to human review."
    return f"Proposed route: {destination}"

```

```
# ---- Step 2: MCP-Style Tool Registry ----
```

```
MCP_TOOLS = {
    "lookup_approved_policy": {
        "function": lookup_approved_policy,
        "safety": "Approved sources only; no real patient information",
        "approval_required": False,
    },
    "estimate_clinic_wait": {
        "function": estimate_clinic_wait,
        "safety": "Teaching estimate only; not an operational prediction",
        "approval_required": False,
    },
    "propose_admin_route": {
        "function": propose_admin_route,
        "safety": "Proposal only; human approval required",
        "approval_required": True,
    },
}
```

```
# ---- Step 3: A2A-Style Agent Cards ----
```

```
# Each card advertises one capability and one allowed MCP tool.
```

```
AGENT_CARDS = {
    "policy-agent": {
        "capability": "approved-policy-lookup",
        "description": "Find source-labeled administrative policy text",
        "allowed_tool": "lookup_approved_policy",
    },
    "wait-time-agent": {
        "capability": "fictional-wait-calculation",
        "description": "Calculate a fictional administrative wait value",
        "allowed_tool": "estimate_clinic_wait",
    },
}
```

```

},
"routing-agent": {
    "capability": "administrative-route-proposal",
    "description": "Propose a non-clinical administrative destination",
    "allowed_tool": "propose_admin_route",
},
}

def discover_agent(capability):
    """Find an approved specialist advertising the needed capability."""
    for agent_name, card in AGENT_CARDS.items():
        if card["capability"] == capability:
            return agent_name
    return None

def send_a2a_task(task):
    """Simulate a traceable handoff from the Coordinator to a specialist."""
    target = task["to_agent"]
    print(f"    A2A → {target}")
    print(f"        Task ID: {task['task_id']}")
    print(f"        Request: {task['message']}")

    if target not in AGENT_CARDS:
        return {
            "status": "needs-human-review",
            "artifact": "No approved specialist was found.",
            "approval_required": True,
        }

    tool_name = AGENT_CARDS[target]["allowed_tool"]
    tool = MCP_TOOLS[tool_name]
    print(f"        MCP → {tool_name}")
    print(f"        Safety: {tool['safety']}")
    result = tool["function"](**task["arguments"])

```

```
return {
    "status": "completed",
    "artifact": result,
    "approval_required": tool["approval_required"],
}
```

---- Step 4: Coordinator Delegation Plan ----

```
def coordinator_agent(request):
    """Delegate all three subtasks and prepare a reviewable draft."""
    plan = [
        (
            "approved-policy-lookup",
            "Find the approved records-request policy.",
            {"question": request["policy_question"]},
        ),
        (
            "fictional-wait-calculation",
            "Calculate the two fictional wait-time values.",
            {
                "current_wait": request["current_wait"],
                "added_minutes": request["added_minutes"],
            },
        ),
        (
            "administrative-route-proposal",
            "Propose an administrative destination.",
            {"request_type": request["request_type"]},
        ),
    ]

    print("📬 COORDINATOR RECEIVED A FICTIONAL REQUEST")
    print(f"    {request['summary']}\n")
    results = []
```

```

for number, (capability, message, arguments) in enumerate(plan, start=1):
    target = discover_agent(capability)
    task = {
        "task_id": f"A2A-{number:03d}",
        "from_agent": "coordinator-agent",
        "to_agent": target or "human-review",
        "message": message,
        "arguments": arguments,
    }
    response = send_a2a_task(task)
    results.append(response)
    print(f"        Result: {response['artifact']}\n")

print("=" * 72)
print("📋 COORDINATOR'S COMBINED DRAFT")
for number, result in enumerate(results, start=1):
    print(f"    {number}. {result['artifact']}")

if any(result["approval_required"] for result in results):
    print("\n🛑 FINAL STATUS: HUMAN REVIEW AND APPROVAL REQUIRED")
else:
    print("\n👤 FINAL STATUS: READY FOR HUMAN REVIEW")

print("    No real PHI was used and no clinical or operational action occurred.")
return results

```

---- Run One Clear Multi-Agent Example ----

```

fictional_request = {
    "summary": (
        "Look up where records requests go, estimate 25 + 15 minutes, "
        "and propose the correct administrative route."
    ),
    "policy_question": "Where do records requests go?",
    "current_wait": 25,

```

```
"added_minutes": 15,  
"request_type": "records",  
}  
  
coordinator_agent(fictional_request)  
  
print("\n💡 KEY INSIGHT")  
print("    A2A connected the Coordinator to specialist agents.")  
print("    MCP connected each specialist to one approved tool.")  
print("    RAG supplied source-labeled policy information.")  
print("    A human still controls any consequential next step.")
```



COORDINATOR RECEIVED A FICTIONAL REQUEST

Look up where records requests go, estimate 25 + 15 minutes, and propose the correct administrative route.

A2A → policy-agent

Task ID: A2A-001

Request: Find the approved records-request policy.

MCP → lookup_approved_policy

Safety: Approved sources only; no real patient information

Result: Based on [POLICY-RECORDS-02]: Records requests are sent to the Health Information Management team.

A2A → wait-time-agent

Task ID: A2A-002

Request: Calculate the two fictional wait-time values.

MCP → estimate_clinic_wait

Safety: Teaching estimate only; not an operational prediction

Result: 40 fictional minutes

A2A → routing-agent

Task ID: A2A-003

Request: Propose an administrative destination.

MCP → propose_admin_route

Safety: Proposal only; human approval required

Result: Proposed route: Health Information Management

=====

 COORDINATOR'S COMBINED DRAFT

1. Based on [POLICY-RECORDS-02]: Records requests are sent to the Health Information Management team.
2. 40 fictional minutes
3. Proposed route: Health Information Management

 FINAL STATUS: HUMAN REVIEW AND APPROVAL REQUIRED

No real PHI was used and no clinical or operational action occurred.

 KEY INSIGHT

A2A connected the Coordinator to specialist agents.
MCP connected each specialist to one approved tool.
RAG supplied source-labeled policy information.
A human still controls any consequential next step.

9. Token and Cost Optimization — Save Cost Without Losing Quality

Why token optimization matters

Many AI services charge for both **input tokens** and **output tokens**. Large prompts, repeated documents, long conversation histories, and unnecessarily long answers can make powerful models expensive. The goal is not to remove useful context—it is to remove waste.

A Simple Cost Mental Model

$$\text{Estimated cost} = \text{input-token cost} + \text{output-token cost}$$

Providers usually publish different rates for each model. More capable models and very long context windows may cost more, so always check the current pricing for the approved service your organization uses.

Fictional example: Sending 100 pages of policies when only two paragraphs answer the question wastes tokens. Retrieval can select just the most relevant approved passages before the model responds.



DO

- **State one clear goal.** Ask for the exact task and audience.
- **Retrieve only relevant approved passages.** Use RAG instead of sending every policy document.
- **Set an output limit.** For example: "Use five bullets and no more than 150 words."
- **Remove repeated instructions.** Keep one short, reusable instruction block.
- **Summarize long histories.** Keep verified decisions and remove old conversational repetition.
- **Use the smallest capable approved model.** Escalate to a larger model when the task truly needs it.
- **Measure usage.** Track input tokens, output tokens, cost per task, and quality.

✗ DON'T

- **Do not paste entire documents by default.** First identify the sections needed.
- **Do not resend the full conversation every time.** Retain only useful, verified context.
- **Do not request long prose when a table or short list works.**
- **Do not repeat examples.** One clear example is often enough.
- **Do not use an expensive model for every simple task.** Classification or formatting may need less capability.
- **Do not remove essential healthcare context only to save money.** Quality and safety come first.
- **Never enter PHI into an unapproved tool.** Fewer tokens do not make private data safe.

Before and After

Wasteful request	Token-optimized request
"Read all 100 pages of our policy library and explain everything."	"Using the two retrieved sections about records requests, provide three routing steps in under 120 words."
"Give me a very detailed response with all possible background."	"Give the answer in five bullets. Include only actions, owner, and approved source."
Re-send the complete chat history for every follow-up.	Provide a short verified summary of prior decisions plus the new question.



QUICK CHECK BEFORE YOU SEND

1. Is the goal clear?
2. Is every included document relevant?
3. Can repeated text be removed?
4. Is the output length specified?
5. Is this the smallest approved model that can safely do the task?
6. Is the content fictional, de-identified, or handled only in an approved PHI environment?

Healthcare safety rule: Optimize waste, not clinical meaning. Never omit information needed for safe interpretation, qualified review, legal compliance, or patient care merely to reduce token cost.

10. AI Governance and Safety — Protecting Patients, Data, and Trust

AI governance is the system of people, policies, decisions, and checks used to oversee AI throughout its lifecycle.

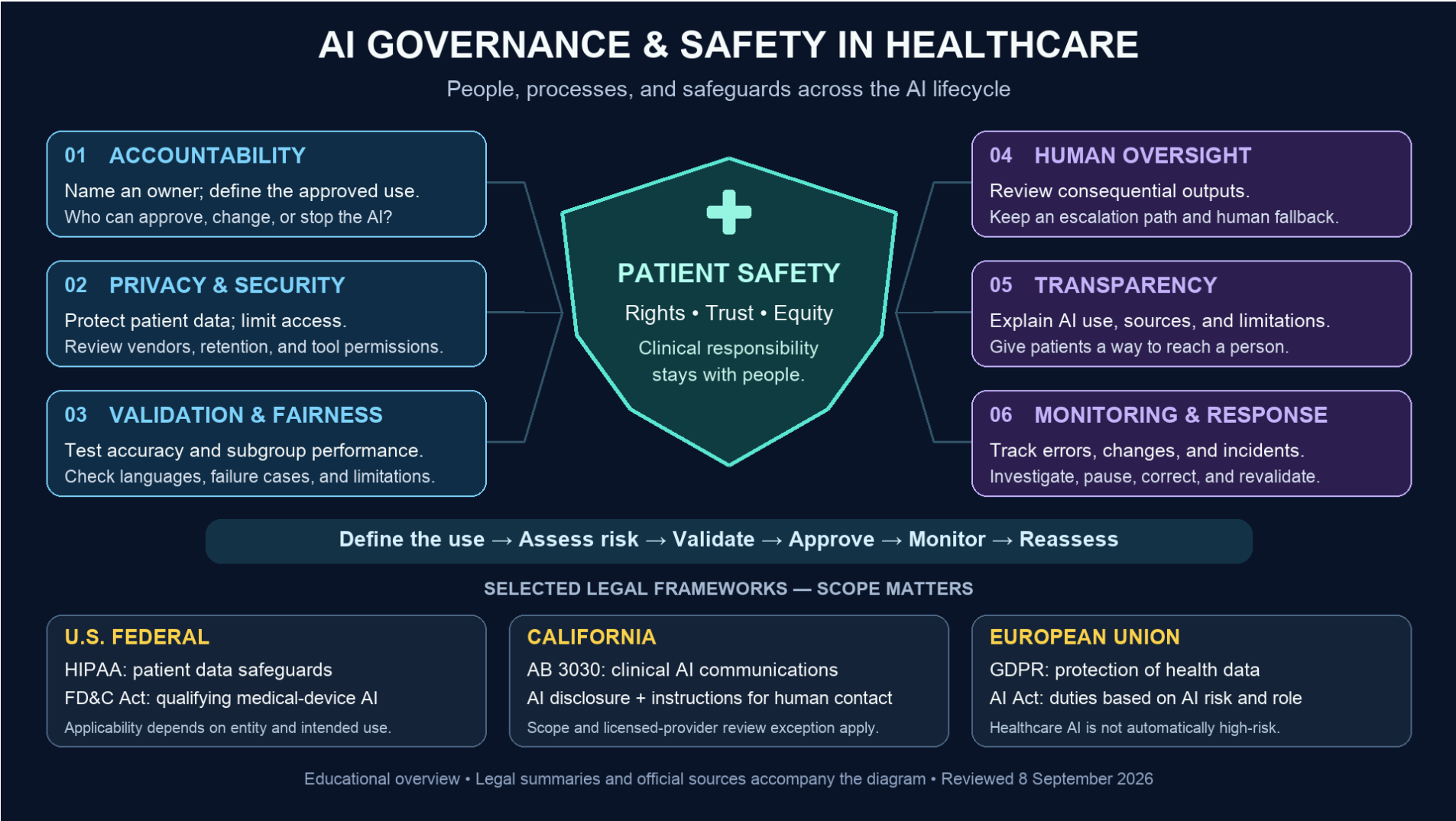
AI safety means reducing the risk that AI causes harm through inaccurate outputs, privacy failures, unfair treatment, misuse, or unsafe actions.

Healthcare connection: The same model can carry very different risks when drafting an appointment reminder, summarizing a clinical note, or supporting a treatment decision. Assess the actual use and its possible impact on patients.

Key idea: A capable model is only one part of a safe healthcare workflow. People remain responsible for approving its use and checking its results.

These teaching principles draw on the [NIST AI Risk Management Framework](#), a voluntary framework rather than a law or a certification of safety.

Healthcare AI Governance — The Big Picture



What Governance Looks Like in Practice

- **Accountability:** Name a clinical or operational owner; document intended use, approval authority, and prohibited uses.
- **Privacy and security:** Use approved systems, minimize patient data, review vendor data use and retention, restrict access, and test malicious instructions in retrieved content.
- **Validation and fairness:** Test on representative cases; compare performance across relevant patient groups and languages.
- **Human oversight:** Specify who reviews consequential outputs, when to escalate, and how to continue safely without AI.
- **Transparency:** Explain relevant AI involvement, limitations, and sources; give patients a way to reach a person.
- **Monitoring and response:** Protect audit records, track failures and model changes, investigate incidents, and pause or revalidate when needed.

A practical workflow: Define the use → Assess risk → Validate → Approve → Monitor → Reassess.

Risk management continues after deployment. This teaching workflow is informed by the [NIST AI RMF](#); it is not a legal compliance checklist.

Selected U.S. Federal Laws

HIPAA and its implementing rules — patient information

For covered entities and business associates, protect PHI through appropriate privacy, security, and breach-response practices. When an AI vendor acts as a business associate, an appropriate business associate agreement (BAA) is needed. A BAA alone does not make a workflow compliant. [HHS guidance](#)

Federal Food, Drug, and Cosmetic Act (FD&C Act) — medical-device software

An AI function intended for medical purposes may fall under FDA device oversight. Classification depends on the function and intended use; some clinical decision-support functions are excluded by statute. FDA guidance explains the distinction but is not itself a new AI law. [FDA guidance](#)

California — AI in Patient Communications

AB 3030 / Health and Safety Code §1339.75 — clinical communications

Specified healthcare organizations using GenAI for patient clinical communications must provide an AI disclosure and instructions for contacting a human, subject to the law's exception for communications read and reviewed by a licensed or certified healthcare provider. Administrative matters such as scheduling and billing are excluded from its definition of patient clinical information. [California law](#)

European Union — Protecting Health Data

General Data Protection Regulation (GDPR) — personal and health data

When GDPR applies, identify a lawful basis under Article 6 and an applicable condition under Article 9 for health data. Apply data minimization, transparency, security, and relevant individual rights. Consent is not the only possible basis, and replacing names does not automatically make data anonymous. [European Commission: legal grounds, data-protection principles](#)

European Union — AI Systems and Medical Devices

EU AI Act — obligations based on risk and role

Assess the system's intended use, risk category, and whether the organization is a provider or deployer. Some medical-device AI is high-risk; healthcare AI is not automatically high-risk. In-scope high-risk requirements include risk management, documentation, logging, human oversight, and performance safeguards. AI literacy and transparency duties also matter where applicable. Requirements have different application dates, so check the current implementation timetable. [European Commission: AI Act](#)

Medical-device connection: Where applicable, the AI Act operates alongside EU medical-device rules; meeting one set of obligations does not replace the other. [European Commission: medical-device / AI Act interplay](#)

Classroom Scenario — A Draft Patient Portal Message

A fictional clinic wants AI to draft a plain-language explanation of clinician-provided discharge text. The draft is for staff review; this workshop does not generate clinical advice or send a patient message.

Before use: Confirm the approved purpose, accountable owner, vendor/data protections, and applicable legal requirements. Test for omitted instructions, invented details, and differences across languages.

Before release: A qualified person compares the draft with the approved source, checks meaning and uncertainty, and follows the organization's review and disclosure policy. A general review step should not be assumed to satisfy every legal exception.

After use: Record approved review information securely, monitor errors and complaints, and reassess after changes. Pause use and escalate if the tool invents clinical information, exposes patient data, or performs poorly for a patient group.

Discussion: Who owns this workflow, who can stop it, and how does the patient reach a human?

Legal scope note: Educational overview reviewed on 8 September 2026. These are selected examples, not a complete legal checklist or legal advice. Applicability depends on jurisdiction, organizational role, data, and intended use; consult the organization's privacy, compliance, and clinical-safety teams for actual deployment.

11. Safe Prompting, Privacy, and Output Review

A clear prompt states the task, the allowed source, the audience, and the required format.

Vague: "Summarize this."

Clearer: "Using only the fictional note below, create three plain-language bullets for staff training. Mark missing information as unknown. Do not add a diagnosis or recommendation."

Before using any approved AI system:

1. Check whether the content contains PHI or other sensitive data.
2. Use the minimum necessary information.
3. Confirm the tool is approved for that data.
4. Require the output to cite supplied sources and state uncertainty.
5. Have an accountable healthcare professional verify the result.

Safety check: A simple detector can miss identifiers. It is only a classroom demonstration, not a privacy solution.

```

def basic_privacy_screen(text):
    """Detect a few obvious placeholder identifiers for classroom use only."""
    patterns = {
        "email": r"\b[\w.-]+@[\w.-]+\.\w+\b",
        "phone-like number": r"\b\d{3}[- ]\d{3}[- ]\d{4}\b",
        "medical record number label": r"\bMRN\s*[:#]?\s*\d+\b",
    }
    return [label for label, pattern in patterns.items() if re.search(pattern, text, re.I)]

def review_output(text, source_label=None):
    """Apply a tiny review checklist to a draft response."""
    return {
        "has_source": bool(source_label),
        "states_uncertainty": "do not know" in text.lower() or "unknown" in text.lower(),
        "requires_human_review": True,
        "clear_length": len(text.split()) <= 40,
    }

safe_example = "Patient A requests a fictional appointment change."
unsafe_example = "Send details to example.person@example.org"

for example in (safe_example, unsafe_example):
    findings = basic_privacy_screen(example)
    print(example)
    print("BLOCK" if findings else "No simple pattern found", findings)
    print()

sample_draft = "I do not know based on the approved sources."
print("Output checklist:", review_output(sample_draft, source_label="POLICY-LIBRARY"))
print("Professional verification is still required.")

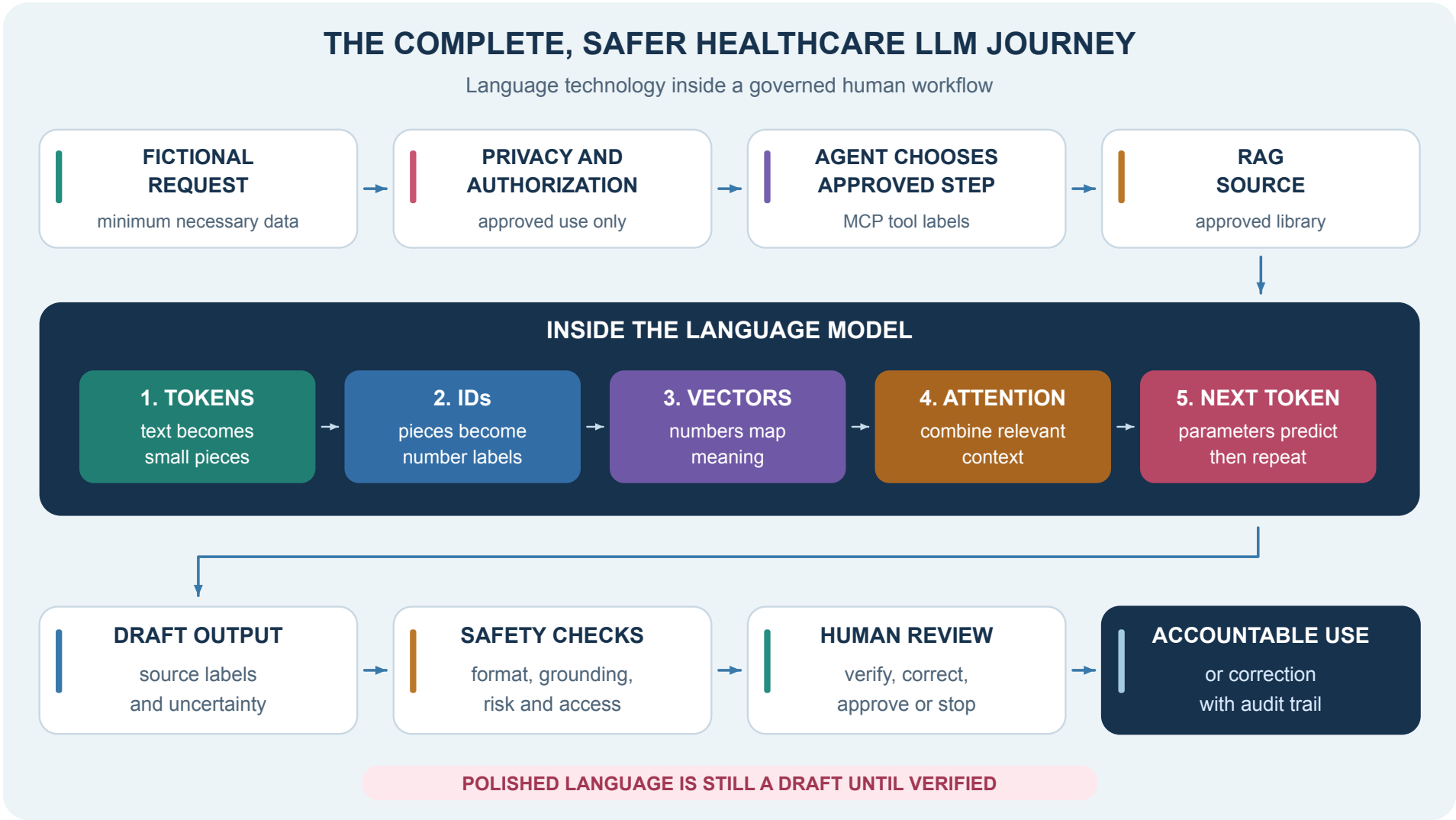
```

Patient A requests a fictional appointment change.
No simple pattern found []

Send details to example.person@example.org
BLOCK ['email']

Output checklist: {'has_source': True, 'states_uncertainty': True, 'requires_human_review': True, 'clear_length': True}
Professional verification is still required.

12. The Full Picture



Concept	Simple healthcare meaning
Tokens	Small text pieces from a fictional message or note
Token IDs	Number labels for those pieces
Embeddings	Numeric representations used to compare meaning
Attention	Context-dependent focus between tokens
Parameters	Learned numeric settings used for predictions
Next-token prediction	Repeatedly choosing likely text
Training	Adjusting parameters to reduce error on examples
RAG	Adding retrieved approved sources to a prompt
Agent	Choosing a tool or step for a task
MCP	A standard description of available tools

Remember: A polished answer is still a draft until its facts, source, context, privacy, and intended use are checked.

13. Five Mini Challenges

Use fictional data only.

1. **Tokens:** Change the appointment sentence and compare the token count.
2. **Parameters:** Change the toy `weight` and `bias`. Observe how the non-clinical output changes.
3. **Similarity:** Add “reschedule my appointment” to the phrase list. Which phrase is most similar in the word-count demo?
4. **Attention:** Add a word to the fictional sentence and assign it a hand-written attention score.
5. **RAG and safety:** Add a fictional, labeled administrative policy. Ask one question it answers and one it cannot answer. Check that the second response says it does not know.

Discussion: For each challenge, identify where privacy screening, source verification, bias review, access controls, audit logging, and human approval belong.

14. Final Takeaway

LLMs can help draft, organize, retrieve, and transform language. They can also hallucinate, omit context, expose sensitive information, and reproduce bias. Healthcare professionals remain accountable for how outputs are checked and used.

Before using an LLM in healthcare

- ☐ **Protect PHI:** Use only approved tools and the minimum necessary data.
- ☐ **Verify sources:** Check that citations are real, current, applicable, and accurately represented.
- ☐ **Expect errors:** Fluency is not evidence; test edge cases and uncertainty.
- ☐ **Check bias:** Evaluate performance across relevant populations and workflows.
- ☐ **Keep humans accountable:** Require qualified review for clinical or consequential use.
- ☐ **Control access:** Apply role-based permissions and least privilege.
- ☐ **Keep an audit trail:** Record sources, tool calls, reviews, and approvals as required by policy.

Final safety message: This notebook is educational only. It does not provide medical advice and must not be used for diagnosis, treatment, medication dosing, or real patient care.